

Complete Deployment Explanation - From Scratch

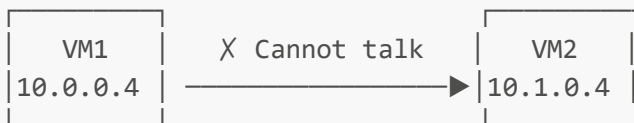
What You Started With

You had:

1. Two VMs created on Azure Cloud
 - VM1 (MyVm) - Frontend Server
 - VM2 (vm2) - Backend Server
2. Global VPC Peering already configured between them
3. FitTrack Pro application code on GitHub
 - Frontend: HTML, CSS, JavaScript
 - Backend: Node.js, Express.js
 - Database: MongoDB

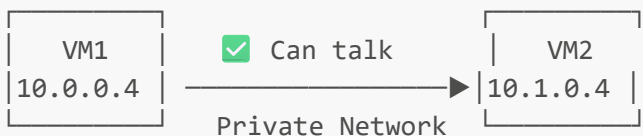
What is VPC Peering?

WITHOUT Peering:



They are in different networks - blocked!

WITH Peering:



Now they can communicate using private IPs!

Why Peering?

- Secure: Traffic never goes to public internet
- Fast: Direct private network communication
- Free: No data transfer costs on private network
- Safe: Backend not exposed to public internet

VM2 - What You Did (Backend Setup)

VM2 History Explained Line by Line

```
# FIRST THING - Test if peering works
ping 10.0.1.4
```

Why: Before doing anything, verify VM1 and VM2
can talk to each other
If ping fails = peering not working
If ping works = we can proceed

```
# Check Ubuntu version
lsb_release -a
```

```
# Check current user
whoami
```

```
# Switch to root user
sudo su
whoami
```

Why:

- lsb_release tells us Ubuntu version
Important because package commands differ
between Ubuntu versions
- whoami confirms who we are logged in as
Regular user = limited permissions
Root = full permissions (dangerous but needed for setup)

```
# Update and upgrade system packages
sudo apt update && sudo apt upgrade -y
```

What it does:

- apt update → Downloads latest package list
(like refreshing app store)
- apt upgrade → Installs latest security patches
- -y flag → Auto answer YES to all prompts

Why: Always update before installing anything
Prevents conflicts and security issues

```
# Install Node.js version 18
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt install nodejs -y
node -v
npm -v
```

What it does:

- curl downloads NodeSource setup script
- Script adds Node.js 18 repository to Ubuntu
- Then we install nodejs package
- node -v and npm -v verify installation

Why Node.js 18:

- FitTrack requires Node.js v14 or higher
- Version 18 is LTS (Long Term Support)
- LTS = stable, supported for years
- Never use latest unstable versions in production

What is npm:

- Node Package Manager
- Like pip for Python, composer for PHP
- Downloads and manages JavaScript libraries

```
# Install Git
sudo apt install git -y
git --version
```

Why:

- Git is needed to clone the application code from GitHub to the VM
- Without Git we cannot download the code

```
# Install PM2 globally
sudo npm install -g pm2
pm2 --version
```

What is PM2:

PM2 = Process Manager 2

The Problem without PM2:

- You start Node.js: `node app.js`
- You close terminal → App DIES ❌
- Server crashes → App DIES ❌
- No auto restart → App stays DEAD ❌

With PM2:

- App runs in background always ✅
- Terminal closes → App keeps running ✅
- App crashes → PM2 restarts it automatically ✅
- Server reboots → PM2 starts app automatically ✅

-g flag = install globally

Available system-wide, not just one project

```
# Add MongoDB repository key (security verification)
curl -fsSL https://www.mongodb.org/static/pgp/server-7.0.asc | \
sudo gpg -o /usr/share/keyrings/mongodb-server-7.0.gpg --dearmor

# Verify the key was saved
ls -l /usr/share/keyrings/mongodb-server-7.0.gpg
```

What is this:

- MongoDB is not in default Ubuntu repository
- We need to add MongoDB's official repository
- GPG key = digital signature
- Verifies packages come from MongoDB officially
- Prevents installing fake/malicious packages

Why MongoDB 7.0:

- Latest stable version
- FitTrack requires MongoDB 4.4+
- Always use newer stable version

```
# Add MongoDB repository to Ubuntu sources
echo "deb [ arch=amd64,arm64 \
signed-by=/usr/share/keyrings/mongodb-server-7.0.gpg ] \
https://repo.mongodb.org/apt/ubuntu jammy/mongodb-org/7.0 multiverse" \
| sudo tee /etc/apt/sources.list.d/mongodb-org-7.0.list

# Update package list to include MongoDB
sudo apt update
```

```
# Install MongoDB
sudo apt install -y mongodb-org
```

What it does:

Step 1: Adds MongoDB download URL to Ubuntu's list of known repositories

Step 2: apt update refreshes package list
Now Ubuntu knows MongoDB exists

Step 3: apt install downloads and installs MongoDB

Think of it like:

- Adding a new app store to your phone
- Then downloading an app from that store

```
# Start MongoDB service
sudo systemctl start mongod

# Enable MongoDB to start on boot
sudo systemctl enable mongod

# Check if MongoDB is running
sudo systemctl status mongod
```

What each command does:

start → Starts MongoDB right now
enable → Makes MongoDB start automatically every time server reboots
status → Shows if MongoDB is running or not

mongod = MongoDB Daemon

Daemon = Background service that runs continuously

systemctl = Tool to manage system services
start/stop/restart/enable/disable/status

```
# Open MongoDB shell to verify
mongosh
```

What is mongosh:

- MongoDB Shell
- Command line interface to MongoDB
- Used to verify database is working
- Can run database queries directly

Why we tested it:

- Confirms MongoDB is installed correctly
- Confirms we can connect to database
- Before running the app, verify DB works

```
# Create application directory
```

```
sudo mkdir -p /opt/fittrack
```

```
# Give ownership to vignesh user
```

```
sudo chown vignesh:vignesh /opt/fittrack
```

```
# Go to that directory
```

```
cd /opt/fittrack
```

Why /opt/fittrack:

- /opt = Optional software directory
- Linux standard for third-party applications
- Keeps application separate from system files
- Easy to find and manage

mkdir -p = Create directory and all parent dirs

chown = Change ownership

vignesh:vignesh = user:group

Nginx/PM2 runs as this user

Must own the files to read them

```
# Clone the application from GitHub
```

```
git clone https://github.com/M-VIGNESH3/Fitness_Tracker.git .
```

```
# The dot (.) means clone INTO current directory
```

```
# Without dot it creates a new subfolder
```

What git clone does:

- Downloads entire repository from GitHub
- Includes all files, folders, history
- Creates exact copy of the code on VM2

After clone we verified:

```
ls -la          → See all files
ls -la server/  → See backend files
ls -la public/  → See frontend files
```

Read package.json files to understand the app

```
cat package.json
cat server/package.json
cat server/app.js
cat server/datadog.js
```

Why we read these files:

package.json = App's identity card

- Lists all dependencies needed
- Lists all available commands
- Shows app name, version

server/app.js = Main backend file

- Express server setup
- MongoDB connection
- API routes
- Port configuration

Always read the code before running it!
Understand what it needs before installing

Install root dependencies

```
cd /opt/fittrack
npm install
```

Install server dependencies

```
cd /opt/fittrack/server
npm install
```

What npm install does:

- Reads package.json
- Downloads all listed dependencies
- Creates node_modules folder
- Like pip install -r requirements.txt in Python

Why twice:

- Root package.json has some dependencies

- server/package.json has different dependencies
- Both need to be installed separately

See what environment variables are needed

```
cat .env.example
```

Create actual environment file

```
nano .env
```

```
cat .env
```

What is .env file:

- Environment Variables file
- Stores configuration that changes per environment
- Never committed to Git (security!)
- Contains:

```
PORT=5000
MONGODB_URI=mongodb://localhost:27017/fitness-tracker
NODE_ENV=production
```

Why .env.example exists:

- Template showing what variables are needed
- Safe to commit to Git (no real values)
- Developers copy it and fill real values

Why not hardcode in code:

- Different values for dev/staging/production
- Secrets like passwords never in code
- Easy to change without touching code

Start the backend with PM2

```
pm2 start server/app.js --name fittrack-backend
```

Check PM2 status

```
pm2 status
```

Check application logs

```
pm2 logs fittrack-backend --lines 20
```

Test if backend responds

```
curl http://localhost:5000
```

```
pm2 start server/app.js
```

→ Runs Node.js app using PM2


```
--name fittrack-backend
  → Gives it a friendly name
  → Easy to manage: pm2 restart fittrack-backend
```

```
pm2 status
  → Shows all running processes
  → Status: online/stopped/errored
  → CPU and memory usage
```

```
pm2 logs
  → Shows application output
  → Error messages
  → Database connection status
```

```
curl http://localhost:5000
  → Tests if backend is actually responding
  → Like visiting the URL in browser
```

```
# Save PM2 process list
pm2 save

# Generate startup script
pm2 startup

# Run the generated command
sudo env PATH=$PATH:/usr/bin \
/usr/lib/node_modules/pm2/bin/pm2 startup systemd \
-u vignesh --hp /home/vignesh
```

Why pm2 save:

- Saves current list of running apps
- PM2 remembers what to restart after reboot

Why pm2 startup:

- Generates a systemd service for PM2
- systemd = Ubuntu's service manager
- Makes PM2 itself start on boot
- Then PM2 starts your saved apps

Without this:
Server reboots → PM2 doesn't start → App is down ❌

With this:
Server reboots → systemd starts PM2 →
PM2 starts fittrack-backend → App is up ✅

VM1 - What We Did Together (Frontend Setup)

Verified Files Were There

```
ls -la /var/www/fittrack
ls -la public/
ls -la public/js/
ls -la public/css/
```

Why:

- Git repo was already cloned on VM1
- Verified all frontend files exist
- public/js/app.js → Main JavaScript
- public/css/main.css → Main stylesheet
- public/pages/ → HTML pages

Checked VM1 Network Identity

```
ip addr show
hostname -I
```

Results:

```
VM1 Private IP = 10.0.0.4
VM1 Public IP  = 20.197.32.213
```

```
Private IP = Used for VM1↔VM2 communication
Public IP  = Used for users to access the app
```

Tested VPC Peering to VM2

```
ping -c 4 10.1.0.4
curl -I http://10.1.0.4:5000
curl http://10.1.0.4:5000
```

Results:

- ✓ Ping = 0% packet loss
- ✓ HTTP = 200 OK
- ✓ HTML = FitTrack homepage returned

This confirmed:

1. Peering works (ping success)
2. Backend reachable (HTTP 200)
3. App is running on VM2 (HTML returned)

Checked API URLs in Frontend Code

```
grep -n "localhost\|fetch\|http\|api" \  
    /var/www/fittrack/public/js/app.js  
  
grep -rn "localhost\|5000\|fetch\|api" \  
    /var/www/fittrack/public/pages/
```

Found:

API_BASE_URL: '/api/auth' ← Relative URL 

This was GREAT news because:

Relative URL '/api/auth'

= Browser sends to same server (VM1 Nginx)

= Nginx forwards to VM2 backend

= No file changes needed!

If it was absolute 'http://localhost:5000/api'

= Would only work on same machine

= We would need to edit every file

Verified and Configured Nginx

```
# Nginx was already installed  
sudo systemctl status nginx # already running   
  
# Created configuration file  
sudo nano /etc/nginx/sites-available/fittrack  
  
# Removed default config  
sudo rm /etc/nginx/sites-enabled/default  
  
# Enabled our config  
sudo ln -s /etc/nginx/sites-available/fittrack \  
    /etc/nginx/sites-enabled/fittrack  
  
# Tested config  
sudo nginx -t
```

```
# Applied config
sudo systemctl reload nginx
```

The Nginx Config We Created - Explained

```
server {
    listen 80;
    # Listen on port 80 (standard HTTP port)
    # Users type http://20.197.32.213 = port 80

    server_name _;
    # _ means match ANY domain or IP
    # Works whether user types IP or domain name

    root /var/www/fittrack/public;
    # Where our frontend files are stored
    # Nginx looks here for HTML, CSS, JS files

    index index.html;
    # Default file to serve for directories

    access_log /var/log/nginx/fittrack_access.log;
    # Records every request that comes in
    # Who accessed what and when

    error_log /var/log/nginx/fittrack_error.log;
    # Records all errors
    # Essential for debugging problems

    location = / {
        try_files /pages/index.html =404;
    }
    # When user visits http://20.197.32.213/
    # Serve the file at public/pages/index.html
    # = / means EXACT match of root URL only

    location / {
        try_files $uri $uri/ =404;
    }
    # For all other URLs like:
    # /css/main.css → serve public/css/main.css
    # /js/app.js → serve public/js/app.js
    # /pages/login.html → serve that file

    location /api/ {
        proxy_pass http://10.1.0.4:5000;
        # THIS IS THE KEY PART!
        # When URL starts with /api/
        # Don't serve a file
        # Instead FORWARD the request to VM2!
    }
}
```

```
    proxy_set_header Host $host;
    # Tell backend the original hostname

    proxy_set_header X-Real-IP $remote_addr;
    # Tell backend the real user IP
    # (not Nginx's IP)

    proxy_connect_timeout 60s;
    proxy_read_timeout 60s;
    # Wait up to 60 seconds for backend response
}
}
```

Debugged the 403 Error

Problem: `curl -I http://localhost` → 403 Forbidden

Error log said:

directory index of "/var/www/fittrack/" is forbidden

Root Cause:

First version of config had wrong root path:

root /var/www/fittrack; ← WRONG ❌

Correct path is:

root /var/www/fittrack/public; ← CORRECT ✅

Why 403 and not 404:

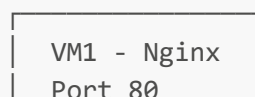
- Directory exists but has no index.html directly
- Nginx won't show directory listing (security)
- So it returns 403 Forbidden

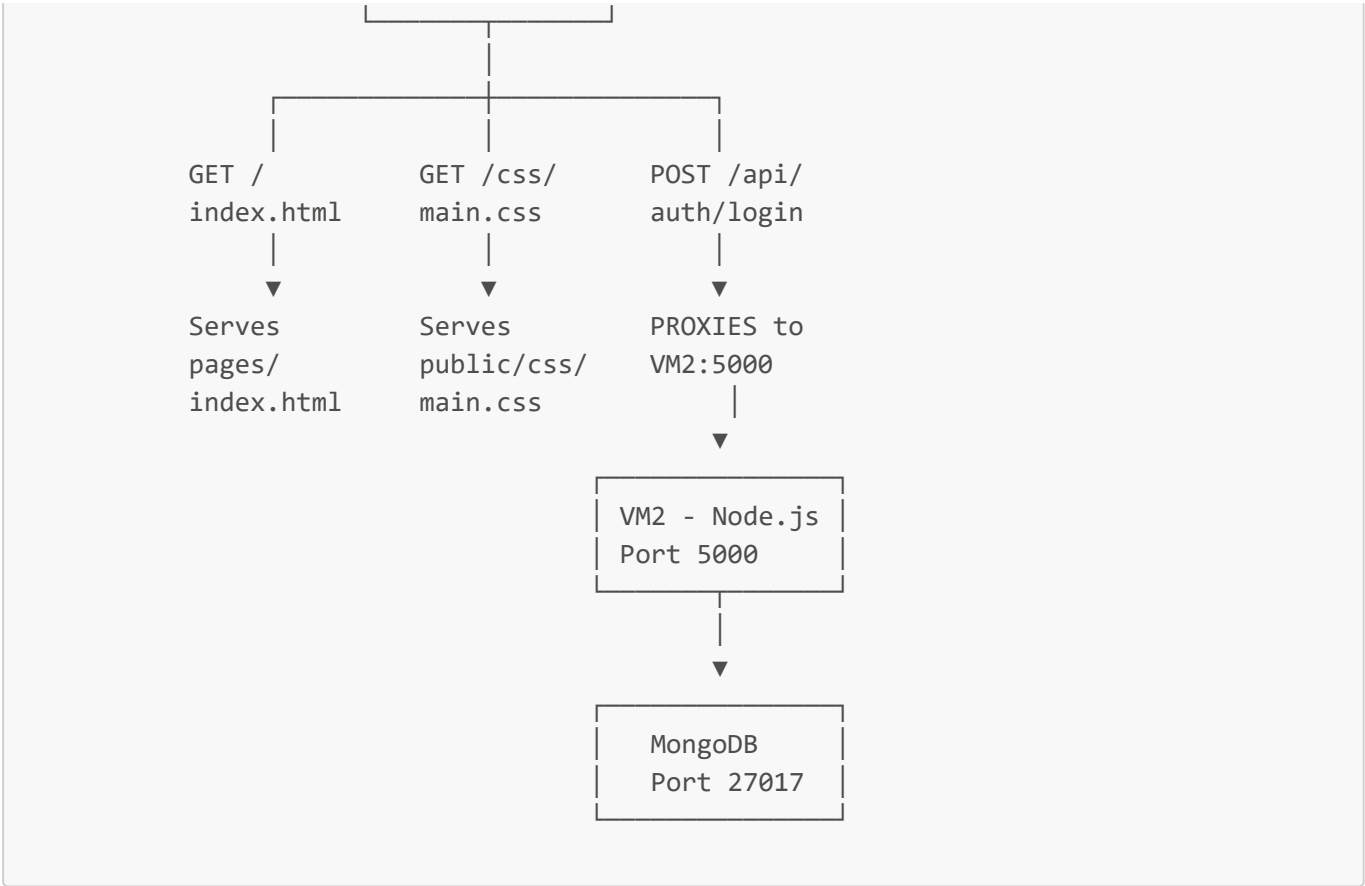
Fix:

- Corrected root path in nginx config
- Tested with `nginx -t`
- Reloaded nginx
- Verified with `curl`

Complete Picture - How Everything Works Together

USER opens browser: `http://20.197.32.213`





Technologies Used and Why

Technology	Where	Why Used
Nginx	VM1	Web server + Reverse Proxy
Node.js	VM2	JavaScript runtime for backend
Express.js	VM2	Web framework for API
MongoDB	VM2	Database to store user data
PM2	VM2	Keep Node.js running always
VPC Peering	Azure	Secure VM1↔VM2 communication
Git	Both	Download code from GitHub

Final Status

VM2 (Backend)

✓

Node.js 18

installed

✓

MongoDB 7.0

installed and running

✓

PM2

managing the app

✓

fittrack-backend

online and healthy

✓

Port 5000

accepting connections

✓

PM2 startup

configured for reboots

VM1 (Frontend)

- ✓ Nginx 1.24 installed and running
- ✓ Frontend files served from /var/www/fittrack/public
- ✓ API proxy forwarding /api/ to VM2:5000
- ✓ Port 80 accepting connections
- ✓ Public IP 20.197.32.213 accessible

Access the app at: <http://20.197.32.213> 🚀

You have successfully deployed a production two-tier application using industry standard practices! 🚀